

The user repository is:

```
package com.jloka.jloka.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.jloka.jloka.models.User;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    User findByUsername(String username);
    User findByEmail(String email);
}
```

---

The tag repository is:

```
package com.jloka.jloka.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.jloka.jloka.models.Tag;

@Repository
public interface TagRepository extends JpaRepository<Tag, Long> {
    public Tag findByName(String name);
}
```

---

```
package com.jloka.jloka.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.jloka.jloka.models.Role;

@Repository
public interface RoleRepository extends JpaRepository<Role, Long> {
    Role findByName(String name);
}
```

---

The post repository is:

```
package com.jloka.jloka.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.jloka.jloka.models.Post;

@Repository
public interface PostRepository extends JpaRepository<Post, Long>{
}
```

---

The comment repository is:

```
package com.jloka.jloka.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.jloka.jloka.models.Comment;

@Repository
public interface CommentRepository extends JpaRepository<Comment, Long> {
}
```

---

The data service is:

```
package com.jloka.jloka.services;

import com.jloka.jloka.models.*;
import com.jloka.jloka.repositories.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.lang.NonNull;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDate;

@Service
public class DataService {

    @Autowired
    private UserRepository userRepository;
```

```

@Autowired
private PostRepository postRepository;

@Autowired
private RoleRepository roleRepository;

@Autowired
private TagRepository tagRepository;

@Transactional
public void createSampleData() {
    // Create a new user with profile (One-to-One)
    User newUser = new User("new_user", "new@example.com");
    UserProfile profile = new UserProfile("New", "User", LocalDate.of(2000,
1, 1));
    newUser.setProfile(profile);

    // Assign roles (Many-to-Many)
    Role userRole = roleRepository.findByName("USER");
    Role editorRole = roleRepository.findByName("EDITOR");
    newUser.addRole(userRole);
    newUser.addRole(editorRole);
    userRepository.save(newUser);

    // Create a new post (One-to-Many)
    Post newPost = new Post("New Spring Boot Feature", "Exciting new features
in Spring Boot 3.2...", newUser);
    // Add tags (Many-to-Many)
    Tag springTag = tagRepository.findByName("Spring Boot");
    Tag javaTag = tagRepository.findByName("Java");
    newPost.addTag(springTag);
    newPost.addTag(javaTag);
    // Add a comment
    Comment comment = new Comment("Looking forward to trying this!", newPost,
newUser);
    newPost.addComment(comment);
    postRepository.save(newPost);
}

@Transactional(readOnly = true)
public void displayUserData(@NonNull Long userId) {
    userRepository.findById(userId).ifPresent(user -> {
        System.out.println("User: " + user.getUsername());
        System.out.println("Email: " + user.getEmail());
    });
}

```

```

// One-to-One
if (user.getProfile() != null) {
    System.out.println("Profile: " + user.getProfile().getFirstName()
+
        " " + user.getProfile().getLastName());
}

// Many-to-Many: Roles
System.out.println("Roles: ");
user.getRoles().forEach(role -> System.out.println("  - " +
role.getName()));

// One-to-Many: Posts
System.out.println("Posts (" + user.getPosts().size() + "):");
user.getPosts().forEach(post -> {
    System.out.println("  - " + post.getTitle());

    // Many-to-Many: Tags for each post
    System.out.println("    Tags: " +
        String.join(", ", post.getTags().stream()
            .map(Tag::getName)
            .toList()));

    // One-to-Many: Comments for each post
    System.out.println("    Comments (" + post.getComments().size() +
");");

    post.getComments().forEach(comment -> {
        System.out.println("      - " + comment.getContent() +
            " (by " + comment.getUser().getUsername() +
");");
    });
});
});
}
}

```

---

The api controller is:

```
package com.jloka.jloka.controllers;

import com.jloka.jloka.services.DataService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.lang.NonNull;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api")
public class ApiController {
    @Autowired
    private DataService dataService;

    @PostMapping("/create-sample")
    public ResponseEntity<String> createSampleData() {
        dataService.createSampleData();
        return ResponseEntity.ok("Sample data created successfully");
    }

    @GetMapping("/user/{id}")
    public ResponseEntity<String> getUserInfo(@NonNull @PathVariable Long id) {
        // In a real application, you would return a DTO or entity
        dataService.displayUserData(id);
        return ResponseEntity.ok("User data displayed in console");
    }
}
```

\*\*\*\*\*

The pom file for jwt authorization is:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.9</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
```

```
<groupId>com.jloka</groupId>
<artifactId>authorization</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>authorization</name>
<description>Demo project for Spring Boot</description>
<url/>
<licenses>
  <license/>
</licenses>
<developers>
  <developer/>
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>21</java.version>
</properties>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <dependency>
    <groupId>org.mariadb.jdbc</groupId>
    <artifactId>mariadb-java-client</artifactId>
    <scope>runtime</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
```

```
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <scope>runtime</scope>
        <version>0.12.6</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <scope>runtime</scope>
        <version>0.12.6</version>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>
```

---

The dto for request is:

```
package com.jloka.authorization.dto;

import lombok.Data;

@Data
public class AuthRequest {
    private String username;
    private String password;
}
```

---

The dto of token response is:

```
package com.jloka.authorization.dto;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class AuthResponse {
    private String token;
}
```

---

The dto of post:

```
import lombok.AllArgsConstructor;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@AllArgsConstructor
public class PostDTO {
    private Long id;
    private String title;
    private String content;
    private LocalDateTime createdAt;
    private Long userId;
    private String username;
}
```

---



The dto of response post is:

```
package com.jloka.authorization.dto;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class PostResponseDTO {
    private Long id;
    private String title;
    private String content;
    private Long userId;
}
```